GUÍA PRÁCTICA DE APRENDIZAJE

Desarrollo de un sistema de ventas con carrito de compras utilizando PHP, MySQL, HTML y CSS

Construye una tienda virtual que permita registrar usuarios, iniciar sesión, consultar productos, administrar un carrito de compras y registrar ventas.

1. ¿Qué aprenderás?

Al desarrollar este proyecto aprenderás a:

- Organizar una aplicación web utilizando el patrón MVC.
- Conectar PHP con una base de datos MySQL.
- Implementar registro e inicio de sesión.
- Utilizar variables de sesión para mantener información entre páginas.
- Proteger páginas para usuarios autenticados.
- Crear un carrito de compras utilizando $_SESSION.
- Procesar y registrar ventas.
- Aplicar HTML y CSS para diseñar interfaces web.

2. Situación problemática

Como Una pequeña empresa necesita implementar una tienda virtual porque actualmente sus clientes deben acudir presencialmente para conocer sus productos y realizar compras.

desarrollador web, tu equipo deberá construir una aplicación que permita a los clientes registrarse, iniciar sesión, seleccionar productos, agregarlos a un carrito y confirmar sus compras.

Tu misión: desarrollar un sistema de ventas funcional que mantenga la información del usuario y del carrito mediante sesiones de PHP.

3. Herramientas necesarias

Antes de comenzar, verifica que tengas instalados:

- XAMPP (Apache y MySQL).
- Visual Studio Code.
- Navegador web.
- phpMyAdmin.

Crea una carpeta llamada SistemaVentas dentro de:

C:\xampp\htdocs\

Podrás acceder al proyecto desde:

http://localhost/SistemaVentas/

4. Arquitectura del proyecto: MVC

Tu aplicación estará organizada en tres componentes:

MODEL: Consultas y operaciones con la base de datos.

VIEW: Interfaces que utiliza el cliente.

CONTROLLER: Procesa las solicitudes y coordina el sistema.

Ejemplo: al agregar un producto al carrito, la vista envía la solicitud, el controlador la procesa y actualiza la sesión.

Estructura de carpetas

Crea las siguientes carpetas y archivos:

```
SistemaVentas/
├── config/
│   └── database.php
├── models/
│   ├── Usuario.php
│   ├── Producto.php
│   └── Venta.php
├── controllers/
│   ├── AuthController.php
│   ├── ProductoController.php
│   ├── CarritoController.php
│   └── VentaController.php
├── views/
│   ├── auth/
│   │   ├── login.php
│   │   └── registro.php
│   ├── productos/
│   │   └── catalogo.php
│   ├── carrito/
│   │   └── index.php
```

```
|   └── ventas/
|       ├── confirmar.php
|       └── historial.php
├── public/
|   └── css/
|       └── estilos.css
├── index.php
└── logout.php
```

5. Preparación de la base de datos

Ingresa a phpMyAdmin, crea una base de datos llamada bd_ventas y ejecuta el siguiente SQL:

```
CREATE DATABASE bd_ventas;

USE bd_ventas;


CREATE TABLE usuarios (     id INT AUTO_INCREMENT

PRIMARY KEY,     nombre VARCHAR(100) NOT NULL,

correo VARCHAR(100) NOT NULL UNIQUE,     password

VARCHAR(255) NOT NULL,     fecha_registro DATETIME

DEFAULT CURRENT_TIMESTAMP

);


CREATE TABLE productos (     id INT

AUTO_INCREMENT PRIMARY KEY,

nombre VARCHAR(100) NOT NULL,

   descripcion TEXT,     precio

DECIMAL(10,2) NOT NULL,     stock

INT NOT NULL,     imagen

VARCHAR(255)

);
```

```
CREATE TABLE ventas (     id INT AUTO_INCREMENT
PRIMARY KEY,     usuario_id INT NOT NULL,     fecha
DATETIME DEFAULT CURRENT_TIMESTAMP,     total
DECIMAL(10,2) NOT NULL,
   FOREIGN KEY (usuario_id) REFERENCES usuarios(id)
);
CREATE TABLE detalle_venta (     id INT
AUTO_INCREMENT PRIMARY KEY,
venta_id INT NOT NULL,     producto_id
INT NOT NULL,     cantidad INT NOT NULL,
precio DECIMAL(10,2) NOT NULL,
subtotal DECIMAL(10,2) NOT NULL,
   FOREIGN KEY (venta_id) REFERENCES ventas(id),
   FOREIGN KEY (producto_id) REFERENCES productos(id)
);
```

Registra algunos productos de prueba:

```
INSERT INTO productos
(nombre, descripcion, precio, stock)
VALUES
('Teclado mecánico', 'Teclado USB', 120.00, 15),
('Mouse inalámbrico', 'Mouse óptico', 45.00, 20),
('Audífonos', 'Audífonos con micrófono', 80.00, 10),
('Monitor 24 pulgadas', 'Monitor LED', 550.00, 8),
('Webcam HD', 'Cámara para videollamadas', 95.00, 12);
```

6. Módulo 1: Conexión a la base de datos

Primero necesitas que PHP pueda comunicarse con MySQL.

Abre config/database.php y escribe:

```
<?php class Database {     private
$host = "localhost";     private
$dbname = "bd_ventas";
private $user = "root";     private
$pass = "";     public function
conectar() {
      try {
         $conexion = new PDO(
            "mysql:host=$this->host;dbname=$this->dbname;charset=utf8mb4",
            $this->user,
            $this->pass
         );
         $conexion->setAttribute(
            PDO::ATTR_ERRMODE,
            PDO::ERRMODE_EXCEPTION
         );
         return $conexion;        } catch(PDOException
$e) {           die("Error de conexión: " . $e-
>getMessage());
      }
   }
}
```

Comprueba lo aprendido:

1. ¿Qué función cumple PDO?

   PDO permite que PHP se comunique con una base de datos de forma segura, flexible y eficiente.

2. ¿Por qué utilizamos una clase para la conexión?
3. Una clase de conexión nos facilita la organización y reotilizacion y mantenimiento del código haciendo que nuestro sistema sea mas ordenado

4. ¿Qué sucedería si el nombre de la base de datos fuera incorrecto?
   Simplemente no se conectaría y nos aparece un error diciendo que la base de datos no existe.

7. Módulo 2: Manejo de sesiones en PHP

Ejemplo desarrollado paso a paso

Este módulo es fundamental para el proyecto. Antes de continuar, debes comprender cómo funciona una sesión.

7.1 ¿Qué es una sesión?

HTTP es un protocolo que no conserva automáticamente la información entre solicitudes.

Por ejemplo, cuando un usuario inicia sesión, PHP necesita algún mecanismo para recordar que ese usuario ya se autenticó.

Las sesiones permiten almacenar información asociada a un usuario durante su navegación.

En PHP se utiliza:

```
session_start();
```

Y para almacenar información:

```
$_SESSION['nombre'] = 'Carlos';
```

Luego, en otra página PHP puedes recuperar esa información:

```
echo $_SESSION['nombre'];
```

7.2 Crear un archivo para administrar sesiones

Para evitar escribir session_start() y las mismas validaciones en todos los archivos, crearás un archivo reutilizable.

Crea config/sesion.php:

```
<?php if (session_status() ===

PHP_SESSION_NONE) {

  session_start();

}

// Verificar si existe un usuario autenticado

function verificarSesion() {
```

```
    if (!isset($_SESSION['usuario_id'])) {

        header("Location: /SistemaVentas/index.php");
        exit;
    }
}

// Obtener el nombre del usuario function
obtenerNombreUsuario() {

    return $_SESSION['nombre'] ?? 'Invitado';
}
```

Este archivo podrá incluirse en los controladores y páginas que necesiten trabajar con la sesión.

7.3 ¿Cómo se almacena la información?

Después de autenticar correctamente a un usuario, guardarás sus datos:

```
$_SESSION['usuario_id'] = $usuario['id'];

$_SESSION['nombre'] = $usuario['nombre'];

$_SESSION['correo'] = $usuario['correo'];
```

Ahora el sistema puede reconocer al usuario mientras navega.

7.4 Proteger páginas mediante sesiones

Imagina que un cliente intenta ingresar directamente al carrito sin iniciar sesión.

Para evitarlo, al comienzo del controlador correspondiente debes colocar:

```
require_once __DIR__ . '/../config/sesion.php';

verificarSesion();
```

Si el usuario no está autenticado, será enviado al inicio de sesión.

7.5 Cerrar sesión

Crea logout.php:

```
<?php


require_once 'config/sesion.php';


// Eliminar variables de sesión session_unset();


// Destruir la sesión session_destroy();


// Regresar al inicio

header("Location: index.php");

exit;
```

7.6 Práctica de comprensión

Realiza las siguientes acciones:

1. Crea un archivo PHP que inicie una sesión.
2. Guarda tu nombre y correo electrónico en $_SESSION.
3. Crea otro archivo que muestre esos datos.
4. Comprueba que puedes visualizar la información desde diferentes páginas.
5. Cierra la sesión.
6. Intenta consultar nuevamente los datos.

Responde: ¿Por qué los datos de sesión pueden utilizarse en diferentes archivos PHP sin tener que volver a registrarlos?

Porque las sesiones de PHP permiten almacenar información del usuario en el servidor y mantenerla disponible en diferentes archivos PHP mientras la sesión esté activa.

8. Módulo 3: Registro e inicio de sesión

En este módulo implementarás el acceso de los usuarios.

8.1 Modelo Usuario

Crea models/Usuario.php:

```
<?php

require_once __DIR__ . '/../config/database.php';

class Usuario {

    private $db;

    public function __construct() {

        $conexion = new Database();
        $this->db = $conexion->conectar();
    }

    public function registrar($nombre, $correo, $password) {

        $sql = "INSERT INTO usuarios
                (nombre, correo, password)
                VALUES (?, ?, ?)";

        $stmt = $this->db->prepare($sql);

        return $stmt->execute([
            $nombre,
            $correo,
            password_hash($password, PASSWORD_DEFAULT)
```

```
        ]);
    }

    public function buscarCorreo($correo) {

        $sql = "SELECT * FROM usuarios WHERE correo = ?";

        $stmt = $this->db->prepare($sql);
        $stmt->execute([$correo]);

        return $stmt->fetch(PDO::FETCH_ASSOC);
    } }
```

Observa que el modelo contiene las operaciones relacionadas con la tabla usuarios.

8.2 Controlador de autenticación

Crea controllers/AuthController.php:

```
<?php

require_once __DIR__ . '/../models/Usuario.php';
require_once __DIR__ . '/../config/sesion.php';

class AuthController {

    private $usuario;

    public function __construct() {
        $this->usuario = new Usuario();
    }
    public function login($correo, $password) {
```

```
        $usuario = $this->usuario->buscarCorreo($correo);

        if ($usuario && password_verify($password, $usuario['password'])) {

            session_regenerate_id(true);

            $_SESSION['usuario_id'] = $usuario['id'];
            $_SESSION['nombre'] = $usuario['nombre'];
            $_SESSION['correo'] = $usuario['correo'];

            header("Location: index.php?ruta=catalogo");
            exit;

        }

        return "Correo o contraseña incorrectos";
    }

    public function registro($nombre, $correo, $password) {

        if ($this->usuario->buscarCorreo($correo)) {
return "El correo ya está registrado";
        }

        $this->usuario->registrar($nombre, $correo, $password);
        return "Usuario registrado correctamente";
    }
```

}

8.3 Formulario de inicio de sesión

Crea views/auth/login.php:

```
<h2>Iniciar sesión</h2>

<form method="POST" action="index.php?ruta=login">

  <label>Correo electrónico</label>
  <input type="email" name="correo" required>

  <label>Contraseña</label>
  <input type="password" name="password" required>

  <button type="submit">Ingresar</button>

</form>

<a href="index.php?ruta=registro">
  Crear una cuenta
</a>
```

8.4 Formulario de registro

Crea views/auth/registro.php:

```
<h2>Crear cuenta</h2>

<form method="POST" action="index.php?ruta=registro">    <label>Nombre completo</label>
  <input type="text" name="nombre" required>
```

```
  <label>Correo electrónico</label>
  <input type="email" name="correo" required>

  <label>Contraseña</label>
  <input type="password" name="password"
minlength="8" required>

  <button type="submit">Registrarme</button>

</form>
```

Tu tarea: Integra los formularios con el controlador y prueba el registro e inicio de sesión. Implementa también mensajes de error y validaciones de los datos recibidos.

Recuerda que las contraseñas nunca deben guardarse en texto plano.

9. Módulo 4: Catálogo de productos

Ahora implementarás el catálogo para que los usuarios puedan consultar los productos disponibles.

9.1 Modelo Producto

Crea models/Producto.php:

```
<?php

require_once __DIR__ . '/../config/database.php';

class Producto {

  private $db;

  public function __construct() {
    $conexion = new Database();
    $this->db = $conexion->conectar();
```

```
    }

    public function listar() {

        $sql = "SELECT * FROM productos ORDER BY nombre";

        return $this->db->query($sql)->fetchAll(
            PDO::FETCH_ASSOC
        );
    }

    public function buscarPorId($id) {

        $sql = "SELECT * FROM productos WHERE id = ?";

        $stmt = $this->db->prepare($sql);
        $stmt->execute([$id]);

        return $stmt->fetch(PDO::FETCH_ASSOC);
    }
}
```

9.2 Controlador Producto

Crea controllers/ProductoController.php:

```
<?php

require_once __DIR__ . '/../models/Producto.php';

class ProductoController {
```

```
    private $producto;

    public function __construct() {
        $this->producto = new Producto();
    }

    public function catalogo() {

        return $this->producto->listar();
    }
}
```

9.3 Vista del catálogo

Crea views/productos/catalogo.php:

```
<h2>Catálogo de productos</h2>

<p>
    Bienvenido, <?= htmlspecialchars($_SESSION['nombre']) ?>
</p>

<div class="productos">

<?php foreach ($productos as $p): ?>

    <div class="producto">
        <h3>
            <?= htmlspecialchars($p['nombre']) ?>
        </h3>
```

```
<p>
  <?= htmlspecialchars($p['descripcion']) ?>
</p>

<p>
  Precio: S/ <?= number_format($p['precio'], 2) ?>
</p>

<p>Stock: <?= $p['stock'] ?></p>

<form method="POST"
action="index.php?ruta=agregar">

  <input type="hidden"
name="producto_id"
value="<?= $p['id'] ?>">

  <input type="number"
name="cantidad"
value="1" min="1"
max="<?= $p['stock'] ?>"
required>

  <button type="submit">
    Agregar al carrito
  </button>

</form>
```

```
    </div>


<?php endforeach; ?>


</div>
```

Actividad de comprobación:

- Ingresa al catálogo.
- Comprueba que se muestran los productos registrados en MySQL.
- Verifica que el precio y el stock sean correctos.
- Identifica qué función cumple el ciclo foreach.
- Explica por qué utilizamos htmlspecialchars() al mostrar información.

10. Módulo 5: Carrito de compras utilizando sesiones

¡Llegamos a la parte más importante del proyecto!

El carrito permitirá almacenar temporalmente los productos seleccionados por cada usuario.

Para ello utilizaremos:

$_SESSION['carrito']

10.1 Comprender la estructura del carrito

Cuando un cliente agrega un producto, almacenaremos su información en una estructura asociativa.

Por ejemplo:

```
$_SESSION['carrito'] = [
1 => [
      'nombre' => 'Teclado',
      'precio' => 120.00,
      'cantidad' => 2
   ],
```

```
2 => [
        'nombre' => 'Mouse',
        'precio' => 45.00,
        'cantidad' => 1
    ]

];
```

Esto representa dos productos seleccionados por el cliente.

Ejemplo de visualización del carrito

Producto Cantidad Subtotal

Teclado 2 S/ 240.00

Mouse 1 S/ 45.00

Total de compra

S/ 285.00

10.2 Crear el controlador del carrito

Abre controllers/CarritoController.php:

```
<?php

require_once __DIR__ . '/../config/sesion.php';
require_once __DIR__ . '/../models/Producto.php'; class
CarritoController {

    private $producto;

    public function __construct() {
        $this->producto = new Producto();
```

```
        if (!isset($_SESSION['carrito'])) {
            $_SESSION['carrito'] = [];
        }
    }

    // Agregar un producto     public
function agregar($id, $cantidad) {

        $id = filter_var($id, FILTER_VALIDATE_INT);
        $cantidad = filter_var($cantidad, FILTER_VALIDATE_INT);

        if (!$id || !$cantidad || $cantidad < 1) {
return "Datos del producto incorrectos";
        }

        $producto = $this->producto->buscarPorId($id);

        if (!$producto) {           return
"El producto no existe";
        }

        $cantidadActual = $_SESSION['carrito'][$id]['cantidad'] ?? 0;
        $nuevaCantidad = $cantidadActual + $cantidad;

        if ($nuevaCantidad > $producto['stock']) {
return "No hay suficiente stock disponible";
        }
```

```
        $_SESSION['carrito'][$id] = [
            'nombre' => $producto['nombre'],
            'precio' => $producto['precio'],
            'cantidad' => $nuevaCantidad
        ];

        return "Producto agregado correctamente";
    }

    // Eliminar un producto
public function eliminar($id) {

        unset($_SESSION['carrito'][$id]);
    }

    // Vaciar el carrito
public function vaciar() {

        $_SESSION['carrito'] = [];
    }

    // Calcular el total     public
function calcularTotal() {

        $total = 0;

        foreach ($_SESSION['carrito'] as $item) {
```

```
            $total += $item['precio'] * $item['cantidad'];
        }

        return $total;
    }
}
```

Observa cómo utilizamos $_SESSION para mantener los productos seleccionados sin tener que guardarlos inmediatamente en MySQL.

10.3 Crear la vista del carrito

Abre views/carrito/index.php:

```
<h2>Mi carrito de compras</h2>

<?php if (empty($_SESSION['carrito'])): ?>

    <p>Tu carrito está vacío.</p>

<?php else: ?>

    <table border="1" cellpadding="10">

        <tr>
            <th>Producto</th>
            <th>Precio</th>
            <th>Cantidad</th>
            <th>Subtotal</th>
            <th>Acción</th>
        </tr>
```

```
<?php foreach ($_SESSION['carrito'] as $id => $item): ?>

<tr>

  <td>
    <?= htmlspecialchars($item['nombre']) ?>
  </td>

  <td>
    S/ <?= number_format($item['precio'], 2) ?>
  </td>

  <td>
    <?= $item['cantidad'] ?>
  </td>

  <td>
    S/ <?= number_format(
      $item['precio'] * $item['cantidad'], 2
    ) ?>
  </td>

  <td>
    <a href="index.php?ruta=eliminar&id=<?= $id ?>">
      Eliminar
    </a>
  </td>
```

```
        </tr>

        <?php endforeach; ?>

    </table>

    <h3>
        Total: S/ <?= number_format($total, 2) ?>
    </h3>

    <a href="index.php?ruta=vaciar">
        Vaciar carrito
    </a>

    <a href="index.php?ruta=confirmar">
        Confirmar compra
    </a>

<?php endif; ?>

<br>
<a href="index.php?ruta=catalogo">
    Seguir comprando
</a>
```

10.4 Comprueba el funcionamiento

Realiza esta secuencia:

1. Inicia sesión con tu usuario.
2. Ingresa al catálogo.

3. Agrega dos teclados.
4. Agrega un mouse.
5. Ingresa al carrito.
6. Comprueba que aparecen los productos.
7. Verifica que el total sea correcto.
8. Elimina un producto.
9. Vacía el carrito.
10. Vuelve al catálogo y agrega otros productos.

Preguntas para analizar:

- ¿Dónde se encuentran almacenados los productos del carrito?

  se encuentran almacenados en una variable de sesión de PHP, generalmente llamada **$_SESSION['carrito']**

- ¿Por qué no se necesita crear una tabla carrito en la base de datos para esta primera versión?

  Porque los productos del carrito se almacenan temporalmente en las sesiones de PHP ($_SESSION). Esto permite guardar los productos seleccionados por el usuario sin necesidad de almacenarlos directamente en la base de datos.

- ¿Qué ocurre cuando agregas dos veces el mismo producto?

  No aparece como otro producto en la tabla sino se acumula

- ¿Cómo se calcula el total de la compra?

  el total de la compra se calcula mediante el método calcularTotal() dentro del CarritoController

- ¿Por qué es necesario comprobar el stock antes de agregar un producto?

El carrito es solo un borrador temporal en la sesión, pero el stock real está en la base de datos. Si no lo compruebamos al agregar, un usuario podría meter 10 unidades de un producto que solo tiene 2, y el error explotaría al final de la compra.

11. Controlador principal: integración de los módulos

Hasta ahora hemos creado modelos, vistas y controladores. Ahora necesitamos un punto de entrada para coordinar el funcionamiento de la aplicación.

Crea o completa index.php:

```php
<?php

require_once 'config/sesion.php';

$ruta = $_GET['ruta'] ?? 'login';

switch ($ruta) {

    case 'login':

        if ($_SERVER['REQUEST_METHOD'] === 'POST') {

            require_once 'controllers/AuthController.php';

            $auth = new AuthController();

            $mensaje = $auth->login(
                $_POST['correo'],
                $_POST['password']
            );

        }

        if (isset($_SESSION['usuario_id'])) {
header("Location: index.php?ruta=catalogo");
            exit;
        }
```

```
        require 'views/auth/login.php';
    break;


    case 'registro':


        require_once 'controllers/AuthController.php';


        $auth = new AuthController();


        if ($_SERVER['REQUEST_METHOD'] === 'POST') {


            $mensaje = $auth->registro(
    trim($_POST['nombre']),
    trim($_POST['correo']),
                $_POST['password']
            );


        }


        require 'views/auth/registro.php';
    break;



    case 'catalogo':


        require_once 'config/sesion.php';
    verificarSesion();
```

```
    require_once 'controllers/ProductoController.php';        $controller = new
ProductoController();

    $productos = $controller->catalogo();

    require 'views/productos/catalogo.php';
    break;



  case 'agregar':

    verificarSesion();

    require_once 'controllers/CarritoController.php';

    $carrito = new CarritoController();

    if ($_SERVER['REQUEST_METHOD'] === 'POST') {

      $mensaje = $carrito->agregar(
        $_POST['producto_id'] ?? 0,
        $_POST['cantidad'] ?? 0
      );
    }

    header("Location: index.php?ruta=carrito");
    exit;

  case 'carrito':
```

```
        verificarSesion();

        require_once 'controllers/CarritoController.php';

        $carrito = new CarritoController();

        $total = $carrito->calcularTotal();

        require 'views/carrito/index.php';
    break;


    case 'eliminar':

        verificarSesion();

        require_once 'controllers/CarritoController.php';

        $carrito = new CarritoController();

        $carrito->eliminar($_GET['id'] ?? 0);

        header("Location: index.php?ruta=carrito");
        exit;

    case 'vaciar':
```

```
        verificarSesion();

        require_once 'controllers/CarritoController.php';

        $carrito = new CarritoController();

        $carrito->vaciar();

        header("Location: index.php?ruta=carrito");
        exit;

    default:

        http_response_code(404);
echo "Página no encontrada";
}
```

Importante: Este enrutador es una versión didáctica simplificada. Para una aplicación real debes validar todas las entradas, utilizar solicitudes POST para operaciones que modifican información (incluidos eliminar y vaciar), incorporar protección CSRF y evitar mostrar errores internos.

También debes agregar el enlace de acceso al carrito en el catálogo:

```
<a href="index.php?ruta=carrito">
    Ver carrito
</a>
```

12. Módulos que desarrollarás por tu cuenta

Trabajo autónomo del estudiante

Ahora que conoces cómo funcionan los modelos, las vistas, los controladores y las sesiones, desarrollarás los siguientes módulos utilizando los ejemplos anteriores como referencia.

No se te proporcionará el código completo: deberás analizar los ejemplos, identificar qué componentes puedes reutilizar y construir las funcionalidades solicitadas.

Módulo 6. Modificación de cantidades del carrito

Objetivo: Permitir que el cliente pueda modificar las cantidades de los productos que ha seleccionado.

Actualmente, el carrito permite agregar y eliminar productos, pero no modificar directamente sus cantidades.

Tu trabajo:

1. Agregar un formulario para modificar la cantidad de cada producto.
2. Crear un método actualizarCantidad() en CarritoController.php.
3. Comprobar que la cantidad sea un número entero mayor que cero.
4. Consultar el stock disponible.
5. Actualizar la cantidad almacenada en $_SESSION['carrito'].
6. Actualizar los subtotales y el total de compra.

Pistas para desarrollar la solución:

- Recuerda que puedes acceder a un producto utilizando su identificador como índice del arreglo.
- Utiliza $_POST para recibir la nueva cantidad.
- Puedes reutilizar el método buscarPorId() del modelo Producto.
- Comprueba que el producto exista en el carrito antes de modificarlo.
- No permitas cantidades superiores al stock.

Pregunta: ¿Por qué es necesario consultar nuevamente el stock y no confiar únicamente en el valor que se muestra en el formulario?

Módulo 7. Confirmación y registro de ventas

Objetivo: Registrar en MySQL los productos que el cliente ha seleccionado en su carrito.

Cuando el usuario presione «Confirmar compra», el sistema deberá procesar la información almacenada en la sesión y convertirla en una venta.

Tu trabajo:

1. Crear Venta.php en la carpeta models.

2. Crear VentaController.php en la carpeta controllers.
3. Crear una vista llamada confirmar.php.
4. Obtener el identificador del usuario desde $_SESSION.
5. Recuperar los productos almacenados en el carrito.
6. Consultar nuevamente los precios y el stock en MySQL.
7. Calcular el importe total.
8. Registrar la venta.
9. Registrar los detalles de la venta.
10. Actualizar el stock.
11. Vaciar el carrito después de una compra exitosa.

Pistas para desarrollar la solución:

Para registrar una venta necesitarás trabajar con dos tablas:

- ventas: almacena la información general de la compra.
- detalle_venta: almacena los productos que forman parte de esa compra.

Recuerda investigar y utilizar las transacciones de PDO:

```
$conexion->beginTransaction();


// Aquí se realizan las operaciones de la venta.


$conexion->commit();
```

Si ocurre un error durante el proceso, investiga cómo utilizar:

```
$conexion->rollBack();
```

Importante: Una venta solo debe considerarse exitosa cuando se hayan registrado correctamente la cabecera, sus detalles y la actualización del stock.

No debes utilizar los precios enviados desde el formulario ni confiar en los datos del carrito como fuente definitiva: verifica la información en la base de datos.

Módulo 8. Historial de compras

Objetivo: Permitir que los clientes consulten sus compras anteriores.

Tu trabajo:

1. Crear un método en Venta.php que consulte las ventas de un usuario.
2. Crear un método para consultar los detalles de una venta.
3. Crear la vista views/ventas/historial.php.
4. Mostrar el número de venta.
5. Mostrar la fecha.
6. Mostrar los productos adquiridos.
7. Mostrar las cantidades y los subtotales.
8. Mostrar el importe total.

Pistas para desarrollar la solución:

- Utiliza el identificador del usuario almacenado en la sesión.
- Puedes utilizar INNER JOIN para relacionar las tablas.
- Recuerda que un usuario solo debe consultar sus propias compras.
- Puedes reutilizar el diseño de la tabla del carrito.

Ejemplo de las relaciones que necesitarás consultar:

```
SELECT
   v.id,
   v.fecha,
   v.total,
   p.nombre,
dv.cantidad,
dv.precio,
dv.subtotal
FROM ventas v
INNER JOIN detalle_venta dv
   ON v.id = dv.venta_id
INNER JOIN productos p
   ON dv.producto_id = p.id
WHERE v.usuario_id = ?;
```

Pregunta: ¿Por qué es importante utilizar el identificador del usuario autenticado en esta consulta?

Piensa en tu cuenta de banco o tu correo electrónico. Cuando entras, solo ves tus movimientos, no los de los demás.

Pasa lo mismo aquí. Si no usas el ID del usuario que inició sesión, la consulta te traería las compras de todos los clientes de la tienda.

13. Diseño de la interfaz con CSS

Tu sistema debe tener una presentación ordenada y agradable. No basta con que funcione: también debe ser fácil de utilizar.

Crea public/css/estilos.css:

```
* {     box-sizing: border-

box;

}


body {      margin: 0;      font-

family:      Arial,      sans-serif;

background: #f1f5f9;      color:

#1e293b;

}


h2 {     color:

#1d4ed8;

}


form {     background:

white;     padding:

20px;     border-

radius: 10px;     max-
```

```
width: 450px;
margin: 20px auto;
}

input { width: 100%;
padding: 12px; margin: 8px
0 15px; border: 1px solid
#cbd5e1; border-radius:
6px;
}

button { background:
#2563eb; color: white;
padding: 11px 18px;
border: none; border-
radius: 6px; cursor:
pointer;
}

button:hover {
background: #1d4ed8;
}

.productos {
display: grid; grid-
template-columns:
    repeat(auto-fit, minmax(220px, 1fr));
gap: 20px; padding: 20px;
```

```
}

.producto  {         background:  white;
padding: 20px;    border-radius: 12px;
box-shadow: 0 2px 8px #00000012;
}

table {     width: 100%;
border-collapse: collapse;
background: white;
}

th, td {      padding: 12px;
text-align:             center;
border: 1px solid #ddd;
}

th {     background:
#1d4ed8;     color:
white;

}

a {     color: #2563eb;
text-decoration: none;
}

@media (max-width: 600px) {
```

```
  table {       font-
size: 12px;
  } }
```

En las vistas que necesiten los estilos, agrega dentro de la sección correspondiente:

```
<link rel="stylesheet" href="public/css/estilos.css">
```

Tu reto de diseño: personaliza los colores, agrega imágenes a los productos, diseña un menú de navegación y mejora la presentación del carrito.

14. Pruebas funcionales del sistema

Antes de entregar tu proyecto, realiza las siguientes pruebas:

Pruebas de funcionamiento

0 de 18

Puedo registrar un nuevo usuario.

Puedo iniciar sesión con mis credenciales.

El sistema muestra mi nombre cuando estoy autenticado.

No puedo ingresar al catálogo o carrito sin iniciar sesión.

Puedo visualizar los productos disponibles.

Puedo agregar productos al carrito.

Puedo modificar las cantidades.

Puedo eliminar productos del carrito.

El carrito conserva los productos al navegar entre páginas.

El sistema calcula correctamente el total.

No puedo comprar productos sin stock suficiente.

Puedo confirmar una compra.

La venta y sus detalles quedan registrados en MySQL.

El stock disminuye después de una compra.

El carrito se vacía después de una compra exitosa.

Puedo consultar mi historial de compras.

No puedo visualizar las compras de otro usuario.

Puedo cerrar sesión correctamente.

15. Reto final de aplicación

Desafío de programación

La empresa que solicitó el sistema ha decidido ampliar sus funcionalidades.

Cada equipo deberá seleccionar e implementar dos funcionalidades adicionales:

1. Implementar un buscador de productos por nombre.
2. Incorporar categorías de productos.
3. Crear un módulo administrativo para registrar productos.
4. Implementar diferentes roles de usuario.
5. Generar un comprobante de venta.
6. Incorporar un reporte de ventas.
7. Permitir que el usuario recupere su carrito después de cerrar sesión, almacenándolo adicionalmente en MySQL.

Deberán explicar cómo implementaron las funcionalidades seleccionadas y qué archivos del proyecto modificaron.

16. Entregables

Cada equipo deberá presentar:

Entregable	**Descripción**
Sistema web	Aplicación funcionando en XAMPP.
Código fuente	Carpetas organizadas mediante MVC.
Base de datos	Archivo SQL exportado.
Evidencias	Capturas de los módulos funcionando.
Informe breve	Explicación del manejo de sesiones y MVC.
Sustentación	Demostración del funcionamiento del sistema.

17. Evaluación del proyecto

Nota máxima: 20 puntos

Criterio	**Puntaje**

Diseño y estructura de la base de datos	2
Organización de la arquitectura MVC	3
Registro, autenticación y manejo de sesiones	4
Funcionamiento del carrito de compras	4
Registro y procesamiento de ventas	3
Diseño de la interfaz	2
Pruebas y sustentación	2
TOTAL	20

18. Reflexión individual

Responde las siguientes preguntas con tus propias palabras:

1. ¿Qué ventajas tiene utilizar MVC para organizar un sistema web?

Aver imaginemos, un MVC convierte un plato de espagueti (código todo revuelto) en una comida bien servida por partes. Te ahorra dolores de cabeza, facilita el trabajo en equipo y hace que tu sistema sea mucho más fácil de escalar y mantener en el futuro.

2. ¿Cuál es la diferencia entre $_POST, $_GET y $_SESSION?

   GET es lo que se ve en la URL, POST es lo que envías escondido, y SESSION es lo que el servidor recuerda de ti.

3. ¿Por qué el carrito de compras se puede implementar utilizando sesiones?

   Porque el carrito es temporal. No necesita guardarlo en la base de datos hasta que el cliente confirme la compra

4. ¿Qué sucede cuando se destruye una sesión?

Se nos borra toda tu información temporal: dejas de estar conectado y tienes que volver a iniciar sesión y el carrito de compras se vacía por completo. Es como si acabaras de entrar a la página por primera vez.

5. ¿Por qué es importante utilizar password_hash() y password_verify()?

   es como guardar la llave de tu casa en una caja fuerte en lugar de dejarla debajo del tapete

6. ¿Qué diferencia existe entre el carrito almacenado temporalmente en una sesión y una venta registrada en MySQL?

   El carrito en la sesión es como un borrador o una lista de deseos: es temporal, vive en la memoria del servidor mientras navegas, y si cierras el navegador o se destruye la sesión, desaparece. Además, no afecta el inventario real.

7. ¿Qué problemas podrían ocurrir si dos clientes intentan comprar simultáneamente las últimas unidades de un producto?

   Si quedan 2 unidades y dos clientes compran al mismo tiempo, el sistema podría leer "hay 2" para ambos y vender las mismas unidades a los dos. Resultado: a uno de los clientes le quedarían debiendo el producto o la base de datos terminaría con stock negativo

19. Recomendaciones para desarrollar tu proyecto

- No copies el código sin comprenderlo. Analiza qué función cumple cada archivo.
- Respeta la separación entre modelos, vistas y controladores.
- Prueba cada módulo antes de continuar con el siguiente.
- Utiliza consultas preparadas para trabajar con MySQL.
- No confíes en la información enviada desde el navegador.
- Valida los datos tanto en el servidor como en los formularios.
- Recuerda que la información de una sesión es temporal y no reemplaza una base de datos.
- Mantén tu código ordenado y utiliza nombres descriptivos.

Tu objetivo como desarrollador

No se trata solamente de conseguir que aparezca un carrito en pantalla. Debes comprender cómo se relacionan PHP, MySQL, MVC y las sesiones para construir una aplicación web que funcione correctamente.

Al finalizar, deberás ser capaz de explicar y demostrar cómo un usuario inicia sesión, selecciona productos, mantiene su carrito y concreta una compra.